



Java Streams API

Section 1: Learn

Introduction to Streams API

Streams API was introduced in **Java 8** to provide a **declarative and functional approach** to processing data.

- Works on **collections**, arrays, or I/O channels
 - Allows **chaining** of multiple operations
 - Supports **lazy evaluation**, **parallelism**, and **cleaner syntax**
-

Difference Between Collections and Streams

Feature	Collections	Streams
Stores data	Yes	No (just processes data)
Data structure	Eager	Lazy (on-demand processing)
Modification allowed	Yes	No (immutable)
Reusability	Can be reused	Can be consumed only once
Parallelism	Manual setup	Built-in with <code>parallelStream()</code>



Stream Operations

1. Intermediate Operations (lazy)

- Return a **new stream**
- Not executed until terminal operation
- Examples: `filter()`, `map()`, `sorted()`, `distinct()`, `limit()`, `skip()`

2. Terminal Operations (eager)

- Triggers **processing**
- Returns result or performs action
- Examples: `forEach()`, `collect()`, `reduce()`, `count()`, `anyMatch()`

Section 2: Practice

Creating Streams from Collections

```
List<String> names = Arrays.asList("Amit", "Neha", "Ravi");  
Stream<String> nameStream = names.stream();
```

Stream Operations in Action

1. Filter

```
names.stream()  
    .filter(n -> n.startsWith("A"))  
    .forEach(System.out::println);
```



2. Map

```
names.stream()
    .map(String::toUpperCase)
    .forEach(System.out::println);
```

3. Reduce

```
List<Integer> nums = Arrays.asList(1, 2, 3, 4);
int sum = nums.stream().reduce(0, (a, b) -> a + b);
System.out.println("Sum: " + sum);
```

4. Collect

```
List<String> result = names.stream()
    .filter(n -> n.length() > 3)
    .collect(Collectors.toList());
```

Parallel Streams

```
names.parallelStream()
    .forEach(System.out::println);
```

Advantage: Parallel processing using multiple threads improves performance for large datasets.

Section 3: Know More (FAQs)

Can streams modify data?

No. Streams are **read-only** and operate on copies.



Can I use the same stream twice?

No. Once a stream is **consumed**, it cannot be reused.

What is the benefit of using map()?

It transforms each element into something new without changing the original data.

When should I use parallelStream()?

When working with **large datasets** where **multi-core processing** is beneficial.

Do streams replace collections?

No. Streams **process data**, while collections **store data**.